

데이터베이스 (Oracle)

1. Database 구성

• Oracle 설치

- 1) <http://www.oracle.com> 오라클 홈페이지에서 회원가입 한다.
- 2) Oracle Database Express Edition 18c를 다운로드하여 설치한다.
- 3) 설치 시 데이터베이스 암호를 1234로 입력한다.

• 서비스 수동으로 변경

- 1) 시작 -> 설정-> 제어판-> 관리도구 -> 서비스에서 Oracle과 관련된 Service를 찾아본다.
- 2) OracleServiceXE의 속성에서 시작유형을 자동에서 수동으로 변경한다.
- 3) OracleXETNSListener의 속성에서 시작유형을 자동에서 수동으로 변경한다.

• Oracle Database 서버접속

- 1) 시작메뉴 -> Oracle-OraDB18Home1-> SQL Plus를 실행한다.

```
SQL> connect;
Enter user-name: system
Enter password: 1234
SQL> disconnect;
```

- 2) 비밀번호 변경 시 sysdba로 로그인 하여 비밀번호를 변경한다.

```
1) connect
2) Enter user-name: /as sysdba
3) pass: 그냥 엔터
4) ALTER USER system IDENTIFIED BY newpass(새 비밀번호);
```

- 3) 오라클 SID(Session Identification)을 확인한다.

```
SQL> select name from v$database;
```

• 데이터베이스/스키마/사용자 생성하기

전체를 관리하는 최상위 데이터베이스를 CDB(Container DB), CDB 안에 들어있는 개발 독립 데이터베이스가 PDB(Pluggable DB)이다.

방법1. C## 접두사 사용(권장 표준 방식): CDB에 공통 사용자를 생성하는 기본 방식이다. 계정명 앞에 C##을 붙여 생성한다.

```
CREATE USER c##username IDENTIFIED BY password;
```

방법2. C## 접두사 없이 생성: _ORACLE_SCRIPT 세션을 TRUE로 변경하여 오라클 12c 이상에서 옛날 방식으로 계정을 생성한다.

```
ALTER SESSION SET "_ORACLE_SCRIPT" = TRUE;
CREATE USER username IDENTIFIED BY password;
```

방법 3. PDB로 이동 후 생성 (운영 환경 추천)

```
SHOW CON_NAME; -- 현재 데이터베이스 컨테이너 확인
SHOW PDBS; -- 사용할 PDB 데이터베이스 컨테이너 목록 확인
ALTER SESSION SET CONTAINER = XEPDB1; -- 특정 PDB 데이터베이스 컨테이너로 세션 변경 (예: XEPDB1)
CREATE USER username IDENTIFIED BY password; -- 새로운 사용자 생성
```

• 사용자 권한부여/비밀번호 변경/사용자 삭제

```
GRANT CONNECT, RESOURCE, DBA TO username; -- 사용자 권한 부여
SELECT username FROM dba_users; -- 사용자 목록으로 생성된 사용자 확인
ALTER USER username IDENTIFIED BY newpassword; -- 사용자 비밀번호 변경
DROP USER username CASCADE; -- 사용자 및 소유한 모든 객체 삭제
```

2. 학사관리 데이터베이스

- 교수(professors) 테이블의 구조

열이름	데이터형	정보내용
pcode	char(3)	교수번호
pname	varchar(15)	교수이름
dept	varchar(30)	소속 학과
hiredate	date	임용일자
title	varchar(15)	직급
salary	int	급여

- 학생(students) 테이블의 구조

열이름	데이터형	정보내용
scode	char(8)	학생번호(학번)
sname	varchar(15)	학생이름
dept	varchar(30)	소속 학과
year	int	학년
birthday	date	생년월일
advisor	char(3)	지도교수

- 강좌(courses) 테이블의 구조

열이름	데이터형	정보내용
lcode	char(4)	강좌번호
lname	varchar(50)	강좌이름
hours	int	강의 시간 수
room	char(3)	강의실
instructor	char(3)	담당교수(교수번호)
capacity	int	최대수강인원수
persons	int	수강신청인원수

- 수강신청(enrollments) 테이블의 구조

열이름	데이터형	정보내용
lcode	char(4)	강좌번호
scode	char(8)	학생번호
edate	date	수강신청일
grade	int	성적

- ERD 다이어그램 (Oracle)

[파일]-[Data Modeler]-[임포트]-[데이터 디렉터리]-[접속 이름 선택]-[스카마/데이터베이스 선택]-[다음]-[테이블/객체 선택]-[다음]-[완료]-[병합]

- ERD 다이어그램 (MySQL)

[Database]-[Reverse Engineer Database]

교수(professors) 테이블의 데이터

PCODE(교수번호)	PNAME(교수이름)	DEPT(소속 학과)	HIREDATE(임용일)	TITLE(직급)	SALARY(급여)
221	이병렬	전산	75/04/03	정교수	3,000,000
228	이재광	전산	91/09/19	부교수	2,500,000
311	강승일	전자	94/06/09	부교수	2,300,000
509	오문환	건축	92/10/14	조교수	2,000,000

학생(Students) 테이블의 데이터

Scode(학생번호)	SNAME(학생이름)	DEPT(소속학과)	YEAR(학년)	BIRTHDAY(생년월일)	ADVISOR(지도교수 번호)
92414029	서연우	전산	3	73/10/06	228
92414033	김창덕	전산	4	73/10/26	221
92514009	이지행	전자	4	73/11/16	311
92514023	김영명	전자	4	73/08/29	311
92454018	이원구	건축	3	74/09/30	509
95454003	이재영	건축	4	76/02/06	509
95414058	박혜경	전산	4	76/03/12	221
96414404	김수정	전산	3	77/12/22	228

강좌(courses) 테이블의 데이터

LCODE(강좌번호)	LNAME(강좌이름)	HOURS(강의시간수)	ROOM(강의실)	INSTRUCTOR(담당교수번호)	CAPACITY(최대수강인원수)	PERSONS(수강신청인원수)
C301	파일처리론	3	506	221	100	80
C401	데이터베이스	3	414	221	80	80
C421	알고리즘	3	510	228	80	72
C312	자료구조	2	510	228	100	60
E221	논리외로	3	304	311	100	80
A109	한국의건축문화	2	101	509	120	36

수강신청(enrollments) 테이블의 데이터

LCODE(강좌번호)	Scode(학생번호)	EDATE(수강신청일)	GRADE(성적)
C401	92414033	98/03/02	85
C301	92414033	98/03/02	80
C421	92414033	98/03/02	0
C401	95414058	98/03/03	90
C312	95414058	98/03/03	80
C401	92514023	98/03/03	70
C301	92414029	98/03/03	90
C421	92414029	98/03/03	0
C312	92414029	98/03/03	70

- 학사관리 테이블 생성

테이블 관련 SQL

```
-- 테이블구조
DESC 테이블명

-- 사용자 테이블 검색
SELECT table_name FROM user_tables;

-- 테이블 삭제
DROP TABLE 테이블명
```

교수 테이블 생성

```
CREATE TABLE professors(
    pcode CHAR(3) NOT NULL,
    pname VARCHAR(15) NOT NULL,
    dept VARCHAR(30),
    hiredate DATE DEFAULT SYSDATE,
    title VARCHAR(15),
    salary INT DEFAULT 0,
    PRIMARY KEY(pcode)
);
```

학생 테이블 생성

```
CREATE TABLE students(
    scode CHAR(8) NOT NULL,
    sname VARCHAR(15) NOT NULL,
    dept VARCHAR(30),
    year INT DEFAULT 1,
    birthday DATE,
    advisor CHAR(3),
    PRIMARY KEY(scode),
    FOREIGN KEY(advisor) REFERENCES professors(pcode)
);
```

강좌 테이블생성

```
CREATE TABLE courses(
    lcode CHAR(4) NOT NULL,
    lname VARCHAR(50) NOT NULL,
    hours INT,
    room CHAR(3),
    instructor CHAR(3),
    capacity INT DEFAULT 0,
    persons INT DEFAULT 0,
    PRIMARY KEY(lcode),
    FOREIGN KEY(instructor) REFERENCES professors(pcode)
);
```

수강신청 테이블 생성

```
CREATE TABLE enrollments(
    lcode CHAR(4) NOT NULL,
    scode CHAR(8) NOT NULL,
    edate DATE,
    grade INT DEFAULT 0,
    PRIMARY KEY(lcode, scode),
    FOREIGN KEY(lcode) REFERENCES courses(lcode),
    FOREIGN KEY(scode) REFERENCES students(scode)
);
```

• 학사관리 데이터 입력

교수 테이블 데이터 입력

```
INSERT INTO professors(pcode,pname,dept,hiredate,title,salary) VALUES('221','이병렬','전산','75/04/03','정교수',3000000);
INSERT INTO professors(pcode,pname,dept,hiredate,title,salary) VALUES('228','이재광','전산','91/09/19','부교수',2500000);
INSERT INTO professors(pcode,pname,dept,hiredate,title,salary) VALUES('311','강승일','전자','94/06/09','부교수',2300000);
INSERT INTO professors(pcode,pname,dept,hiredate,title,salary) VALUES('509','오문환','건축','92/10/14','조교수',2000000);
```

학생 테이블 데이터 입력

```
INSERT INTO students(scode,sname,dept,year,birthday,advisor) VALUES('92414029','서연우','전산',3,'73/10/06','228');
INSERT INTO students(scode,sname,dept,year,birthday,advisor) VALUES('92414033','김창택','전산',4,'73/10/26','221');
INSERT INTO students(scode,sname,dept,year,birthday,advisor) VALUES('92514009','이지행','전자',4,'73/11/16','311');
INSERT INTO students(scode,sname,dept,year,birthday,advisor) VALUES('92514023','김형명','전자',4,'73/08/29','311');
INSERT INTO students(scode,sname,dept,year,birthday,advisor) VALUES('92454018','이원구','건축',3,'74/09/30','509');
INSERT INTO students(scode,sname,dept,year,birthday,advisor) VALUES('95454003','이재영','건축',4,'76/02/06','509');
INSERT INTO students(scode,sname,dept,year,birthday,advisor) VALUES('95414058','박혜경','전산',4,'76/03/12','221');
INSERT INTO students(scode,sname,dept,year,birthday,advisor) VALUES('96414404','김수정','전산',3,'77/12/22','228');
```

강좌테이블 데이터 입력

```
INSERT INTO courses(lcode,lname,hours,room,instructor,capacity,persons) VALUES('C301','파일처리론',3,'506','221',100,80);
INSERT INTO courses(lcode,lname,hours,room,instructor,capacity,persons) VALUES('C401','데이터베이스',3,'414','221',80,80);
INSERT INTO courses(lcode,lname,hours,room,instructor,capacity,persons) VALUES('C421','알고리즘',3,'510','228',80,72);
INSERT INTO courses(lcode,lname,hours,room,instructor,capacity,persons) VALUES('C312','자료구조',2,'510','228',100,60);
INSERT INTO courses(lcode,lname,hours,room,instructor,capacity,persons) VALUES('E221','논리회로',3,'304','311',100,80);
INSERT INTO courses(lcode,lname,hours,room,instructor,capacity,persons) VALUES('A109','한국의건축문화',2,'101','509',120,36);
```

수강신청 테이블 데이터 입력

```
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C401','92414033','98/03/02',85);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C301','92414033','98/03/02',80);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C421','92414033','98/03/02',0);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C401','95414058','98/03/03',90);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C301','95414058','98/03/03',80);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C312','95414058','98/03/03',80);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C401','92514023','98/03/03',70);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C301','92514023','98/03/03',70);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C421','92514023','98/03/03',70);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C301','92414029','98/03/03',90);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C421','92414029','98/03/03',0);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C312','92414029','98/03/03',70);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('E221','92414029','98/03/03',75);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('A109','92414029','98/03/03',90);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C301','92514009','98/03/03',70);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C401','92514009','98/03/03',85);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('E221','92514009','98/03/03',85);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C301','96414404','98/03/04',75);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C401','96414404','98/03/04',75);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C421','96414404','98/03/04',75);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C312','92454018','98/03/04',90);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('E221','92454018','98/03/04',90);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('A109','95454003','98/03/05',85);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('C301','95454003','98/03/05',83);
INSERT INTO enrollments(lcode, scode, edate, grade) VALUES('E221','95454003','98/03/05',75);
```

• 학사관리 데이터베이스 조회

1) 교수 테이블의 모든 행, 모든 컬럼의 데이터를 검색하시오.

```
SELECT * FROM professors;
```

2) 교수 테이블에서 모든 교수의 이름, 학과, 직급 컬럼을 검색하시오.

```
SELECT pname, dept, title FROM professors;
```

3) 교수 테이블에서 교수들이 근무하는 소속 학과를 중복을 제거하여 검색하시오.

```
SELECT DISTINCT(dept) FROM professors;
```

4) 학생 테이블에서 '전산'학과 이면서 3학년 학생들의 학생번호, 학생이름, 학과, 학년 컬럼을 검색하시오.

```
SELECT scode, sname, dept, year FROM students WHERE dept='전산' AND year=3;
```

5) 학생 테이블에서 3학년이거나 4학년 학생들의 학생번호, 학생이름, 학년 컬럼을 검색하시오.

```
SELECT scode, sname, year FROM students WHERE year=3 OR year=4;
```

```
SELECT scode, sname, year FROM students WHERE year IN (3, 4);
```

6) 학생 테이블에서 생년월일이 '1973-01-01'에서 '1973-12-31' 사이의 학생들의 학생번호, 학생이름, 생년월일 컬럼을 검색하시오.

```
SELECT scode, sname, birthday FROM students  
WHERE birthday >= '1973-01-01' AND birthday <= '1973-12-31';
```

```
SELECT scode, sname, birthday FROM students  
WHERE birthday BETWEEN '1973-01-01' AND '1973-12-31';
```

7) 학생 테이블에서 학생번호, 학생이름 컬럼과 생년월일의 포맷을 'YYYY-MM-DD'로 출력하시오.

```
SELECT scode, sname, TO_CHAR(birthday, 'YYYY-MM-DD') FROM students;
```

8) 학생 테이블에서 '73'년도에 태어난 학생들의 학생번호, 학생이름, 생년월일 컬럼을 검색하시오.

```
SELECT scode, sname ,birthday FROM students WHERE TO_CHAR(birthday, 'YY') = '73';
```

9) 학생 테이블에서 '10'월에 태어나지 않은 학생들의 학생번호, 학생이름, 생년월일 컬럼을 검색하시오.

```
SELECT scode, sname, birthday FROM students WHERE TO_CHAR(birthday, 'MM') != '10';
```

```
SELECT scode, sname, birthday FROM students WHERE TO_CHAR(birthday, 'MM') <> '10';
```

10) 학생 테이블에서 성이 '이'인 학생들의 학생번호, 학생이름 컬럼을 검색하시오.

```
SELECT scode, sname FROM students WHERE sname LIKE '이%';
```

11) 강좌 테이블에서 강좌이름에 '건축'이 들어가는 강좌들의 강좌번호, 강좌이름 컬럼을 검색하시오.

```
SELECT lcode, lname FROM courses WHERE lname LIKE '%건축%';
```

• 학사관리 데이터베이스 조인

1) 교수 테이블에 '510', '강감찬', '건축' 데이터를 입력하시오.

```
INSERT INTO professors(pcode, pname, dept, hiredate, salary) VALUES('510', '강감찬', '건축', '95/10/01', 2500000);
```

2) 학생 테이블에 '95454005', '홍길동', '건축' 데이터를 입력하시오.

```
INSERT INTO students(scode, sname, dept) VALUES('95454005', '홍길동', '전산');
```

3) 학생 테이블에서 모든 행에 대한 모든 칼럼과 각 학생의 지도 교수이름, 교수의 학과 칼럼을 검색하시오.

```
SELECT students.*, pname, professors.dept
FROM students, professors WHERE advisor=pcode;
```

```
SELECT students.*, pname, professors.dept
FROM students INNER JOIN professors ON advisor=pcode;
```

```
SELECT students.*, pname, professors.dept
FROM students LEFT JOIN professors ON advisor=pcode;
```

```
SELECT students.*, pname, professors.dept
FROM students RIGHT JOIN professors ON advisor=pcode;
```

4) 강좌 테이블에 'A10', '건축학개론' 데이터를 입력하시오.

```
INSERT INTO courses(lcode, lname) VALUES('A110', '건축학개론');
```

5) 강좌 테이블에서 모든 행에 대한 모든 칼럼과 교수 테이블의 담당 교수이름, 교수의 학과 칼럼을 검색하시오.

```
SELECT courses.*, pname
FROM courses, professors WHERE instructor=pcode;
```

```
SELECT courses.*, pname
FROM courses INNER JOIN professors ON instructor=pcode;
```

```
SELECT courses.*, pname
FROM courses LEFT JOIN professors ON instructor=pcode;
```

```
SELECT courses.*, pname
FROM courses RIGHT JOIN professors ON instructor=pcode;
```

6) 수강신청 테이블의 모든 행에 대한 모든 칼럼과 강좌 테이블의 강좌이름 칼럼을 검색하시오.

```
SELECT e.*, lname
FROM enrollments e, courses c WHERE e.lcode=c.lcode;
```

7) 수강신청 테이블의 모든 행에 대한 강좌번호, 강좌이름, 학생번호, 학생이름, 수강신청일 칼럼을 검색하시오.

```
SELECT e.lcode, c.lname, e.scode, s.sname, edate, grade
FROM enrollments e, courses c, students s
WHERE e.lcode=c.lcode AND e.scode=s.scode;
```

• 학사관리 데이터베이스 집계함수

1) 교수 테이블의 교수 인원수를 출력하시오.

```
SELECT COUNT(*) 인원수 FROM professors;
```

2) 학생 테이블의 학생 인원수를 출력하시오.

```
SELECT COUNT(*) 인원수 FROM students;
```

3) 학생 테이블에서 '전산'과 학생들의 인원수를 출력하시오.

```
SELECT COUNT(*) 인원수 FROM students WHERE dept='전산';
```

4) 학생 테이블의 학생들은 몇 개 학과에 소속되어있는지 출력하시오.

```
SELECT COUNT(DISTINCT(dept)) 학과수 FROM students;
```

5) 교수 테이블의 교수 인원, 급여평균, 급여합계를 출력하시오.

```
SELECT 급여합계, COUNT(*) 인원수, AVG(salary) 급여평균, SUM(salary) FROM professors;
```

```
SELECT TO_CHAR(AVG(salary),'99,999,999') 급여평균, TO_CHAR(SUM(salary),'999,999,999') 급여합계, COUNT(*) 인원수  
FROM professors;
```

6) 학생 테이블에서 학과별 학생 인원수를 출력하시오.

```
SELECT dept, COUNT(*) 학생 인원수 FROM students  
GROUP BY dept;
```

7) 교수 테이블에서 학과별 교수수를 출력하시오.

```
SELECT dept, COUNT(*) 교수 인원수 FROM professors  
GROUP BY dept;
```

8) 학생 테이블에서 학과, 학년별 인원수를 학과, 학년별로 오름차순 정렬하여 출력하시오.

```
SELECT dept, year, COUNT(*) 인원수 FROM students  
GROUP BY dept, year ORDER BY dept, year;
```

9) 학생 테이블에서 지도교수별 지도학생수를 출력하시오.

```
SELECT advisor, COUNT(*) 지도학생수 FROM students  
GROUP BY advisor;
```

```
SELECT advisor, pname, COUNT(*) 지도학생수  
FROM students LEFT JOIN professors ON advisor=pcode  
GROUP BY advisor, pname;
```

10) 강좌 테이블에서 담당교수별 담당과목수를 출력하시오.

```
SELECT instructor, COUNT(*) 담당과목수  
FROM courses  
GROUP BY instructor;
```

```
SELECT instructor, pname, COUNT(*) 담당과목수
FROM courses LEFT JOIN professors ON instructor=pcode
GROUP BY instructor, pname;
```

11) 수강신청 테이블에서 수강신청일별 수강인원수를 출력하시오.

```
SELECT TO_CHAR(edate, 'YYYY-MM-DD') 수강신청일, COUNT(*) 수강인원
FROM enrollments
GROUP BY edate;
```

12) 수강신청 테이블에서 학생별 학생이름, 평균점수, 합계점수, 수강신청한 과목수를 출력하시오.

```
SELECT scode, TO_CHAR(AVG(grade), '999.99') 평균, SUM(grade) 합계, COUNT(*) 과목수
FROM enrollments
GROUP BY scode;
```

```
SELECT e.scode, sname, TO_CHAR(AVG(grade), '999.99') 평균, SUM(grade) 합계, COUNT(*) 과목수
FROM enrollments e, students s WHERE e.scode=s.scode
GROUP BY e.scode, sname;
```

```
SELECT e.scode, sname, TO_CHAR(AVG(grade), '999.99') 평균, SUM(grade) 합계, COUNT(*) 과목수
FROM enrollments e, students s
WHERE e.scode=s.scode
GROUP BY e.scode, sname
HAVING avg(grade) >= 75;
```

```
SELECT e.scode, sname, TO_CHAR(AVG(grade), '999.99') 평균, SUM(grade) 합계, COUNT(*) 과목수
FROM enrollments e, students s
WHERE e.scode=s.scode
GROUP BY e.scode, sname
HAVING AVG(grade) >= 75
ORDER BY AVG(grade) DESC;
```

13) 수강신청 테이블에서 강좌별 강좌이름, 평균점수, 합계점수, 수강신청한 학생수를 출력하시오.

```
SELECT lcode, TO_CHAR(AVG(grade), '999.99') 총점, SUM(grade) 합계, COUNT(*) 과목수
FROM enrollments GROUP BY lcode;
```

```
SELECT e.lcode, lname, TO_CHAR(AVG(grade), '999.99') 총점, SUM(grade) 합계, COUNT(*) 과목수
FROM enrollments e, courses c
WHERE e.lcode=c.lcode
GROUP BY e.lcode, lname;
```

```
SELECT e.lcode, lname, TO_CHAR(AVG(grade), '999.99') 총점, SUM(grade) 합계, COUNT(*) 과목수
FROM enrollments e, courses c
WHERE e.lcode=c.lcode
GROUP BY e.lcode, lname
HAVING AVG(grade) >= 80;
```

```
SELECT e.lcode, lname, TO_CHAR(AVG(grade), '999.99') 총점, SUM(grade) 합계, COUNT(*) 과목수
FROM enrollments e, courses c
WHERE e.lcode=c.lcode
GROUP BY e.lcode, lname
HAVING AVG(grade) >= 80
ORDER BY AVG(grade) DESC;
```

• 학사관리 데이터베이스 서버쿼리

1) 교수 테이블에서 전체 평균 급여보다 많이 받은 교수의 모든 칼럼을 검색하시오.

```
SELECT * FROM professors
WHERE salary > (SELECT AVG(salary) FROM professors);
```

2) 급여(salary)을 가장 많이 받는 교수의 모든 칼럼을 검색하시오.

```
SELECT * FROM professors
WHERE salary = (SELECT MAX(salary) FROM professors);
```

3) 강좌이름에 '건축'이 포함된 강좌를 강의하는 교수들의 모든 칼럼을 검색하시오.

```
SELECT * FROM professors
WHERE pcode IN (SELECT instructor FROM courses WHERE lname LIKE '%건축%');
```

4) 수강생이 한 명도 없는 강의의 모든 칼럼을 검색하시오.

```
SELECT * FROM courses
WHERE lcode NOT IN (SELECT DISTINCT lcode FROM enrollments);
```

5) 3학년 학생을 지도하고 있는 교수들의 모든 칼럼을 검색하시오.

```
SELECT * FROM professors
WHERE pcode IN (SELECT DISTINCT(advisor) FROM students WHERE year=3);
```

6) 강의실 '510'호에서 강의하는 교수들의 모든 칼럼을 검색하시오.

```
SELECT * FROM professors
WHERE pcode IN (SELECT instructor FROM courses WHERE room='510');
```

7) '98/03/04'에 수강신청 한 학생들의 모든 칼럼을 검색하시오.

```
SELECT * FROM students
WHERE scode IN (SELECT scode FROM enrollments WHERE edate = '98/03/04');
```

8) 가장 최근(가장 나중에) 입사한 교수의 모든 칼럼을 검색하시오.

```
SELECT * FROM professors
WHERE hiredate IN (SELECT MAX(hiredate) FROM professors);
```

9) 나이가 가장 어린 학생(생일이 가장 최근인 학생)의 모든 칼럼을 검색하시오.

```
SELECT * FROM students
WHERE birthday IN (SELECT MAX(birthday) FROM students WHERE birthday IS NOT NULL);
```

10) '1973'년생 학생들을 지도하는 교수들의 모든 칼럼을 검색하시오.

```
SELECT * FROM professors
WHERE pcode IN (SELECT DISTINCT(advisor) FROM students where TO_CHAR(birthday, 'YYYY')='1973');
```

11) 학점이 가장 낮게(grade 최솟값) 등록된 수강신청 모든 칼럼을 검색하시오.

```
SELECT * FROM enrollments
WHERE grade IN (SELECT MIN(grade) FROM enrollments WHERE grade > 0);
```

12) 학과별 평균 급여를 계산한 후 평균 급여가 230만 원 이상인 학과만 추출하시오.

```
SELECT *
FROM (SELECT dept, AVG(salary) AS avg_salary FROM professors GROUP BY dept)
WHERE avg_salary >= 2300000;
```

13) 학생 수가 3명 이상인 학과만 추출하시오.

```
SELECT *
FROM (SELECT dept, COUNT(*) AS stu_cnt FROM students GROUP BY dept)
WHERE stu_cnt >= 3;
```

14) 평균 성적이 80점 이상인 학생번호를 추출하시오.

```
SELECT *
FROM (SELECT scode, AVG(grade) stu_avg FROM enrollments GROUP BY scode)
WHERE stu_avg >= 80;
```

15) 평균 성적이 80점 이상인 강좌번호를 추출하시오.

```
SELECT *
FROM (SELECT lcode, AVG(grade) cou_avg FROM enrollments GROUP BY lcode)
WHERE cou_avg >= 80;
```

16) 수강인원이 3명 이상인 강좌번호를 추출하시오.

```
SELECT *
FROM (SELECT lcode, COUNT(*) stu_cnt FROM enrollments GROUP BY lcode)
WHERE stu_cnt >=3;
```

17) 학생 목록 조회 시 학생번호, 학생이름, 지도교수 번호, 지도교수 이름을 추출하시오.

```
SELECT s.scode, s.sname, s.advisor,
(SELECT pname FROM professors WHERE pcode=s.advisor)
FROM students s;
```

18) 과목 목록 조회 시 각 과목의 실제 수강 신청자 수 함께 추출하시오.

```
SELECT c.*,
(SELECT COUNT(*) FROM enrollments e WHERE c.lcode=e.lcode) stu_cnt
FROM courses c;
```

19) 학생 목록 조회 시 각 학생이 수강 신청한 총 과목들의 평균점수를 출력하시오.

```
SELECT s.*,
(SELECT AVG(grade) FROM enrollments e WHERE s.scode=e.scode) avg_grade
FROM students s;
```

20) 강의 목록 조회 시 담당 교수의 이름(pname)과 학과(dept)를 함께 추출하시오.

```
SELECT c.lcode, c.lname, c.instructor,
(SELECT pname FROM professors WHERE pcode=c.instructor) pname,
(SELECT dept FROM professors WHERE pcode=c.instructor) dept
FROM courses c;
```

21) 수강신청 내역에 과목명과 학생이름을 함께 추출하시오.

```
SELECT e.lcode, e.scode,
(SELECT lname FROM courses c WHERE c.lcode=e.lcode),
(SELECT sname FROM students s WHERE s.scode=e.scode)
FROM enrollments e;
```

• 학사관리 데이터베이스 갱신

1) 학생 테이블에 '92514010', '심청이', '전자', '73/10/04' 데이터를 입력하시오.

```
INSERT INTO students(scode, sname, dept, birthday)
VALUES('92514010', '심청이', '전자', '73/10/04');
```

2) 수강신청 테이블에 학생번호 '92514010', 강좌번호 'C301', 'C401' 데이터를 입력하시오.

```
INSERT INTO enrollments(scode, lcode, edate)
VALUES('92514010', 'C301', '98/03/03');
```

```
INSERT INTO enrollments(scode, lcode, edate)
VALUES('92514010', 'C401', '98/03/03');
```

3) 수강신청일이 '1998년 3월 2일'인 데이터를 'enroll980302' 테이블로 복사하시오.

```
CREATE TABLE enroll980302 AS
SELECT lcode, scode, edate, grade FROM enrollments WHERE edate='98/03/02';
```

4) 4학년 학생 데이터를 students4 테이블로 복사하시오.

```
CREATE TABLE students4 AS
SELECT * FROM students WHERE year=4;
```

5) 수강신청 테이블에 학생번호 '92514010', 강좌번호 'C301' 데이터를 삭제하시오.

```
DELETE FROM enrollments WHERE scode='92514010' AND lcode='C301';
```

6) 수강신청 테이블에서 '심청이'의 수강신청 데이터를 모두 삭제하시오.

```
DELETE FROM enrollments WHERE scode IN
(SELECT scode FROM students WHERE sname='심청이');
```

7) 학생 테이블에서 '심청이' 학생의 데이터를 삭제하시오.

```
DELETE FROM students WHERE sname='심청이';
```

8) 위에서 생성한 'enroll980302' 테이블의 모든 데이터를 모두 삭제하고 테이블을 제거한다.

```
DELETE FROM enroll980302;
```

```
DROP TABLE enroll980302;
```

9) 위에서 생성한 'students4' 테이블의 모든 데이터를 모두 삭제하고 테이블을 제거한다.

```
DELETE FROM students4;
```

```
DROP TABLE students4;
```

10) 교수 테이블에서 '오문환' 교수의 직급을 '조교수'에서 '부교수'로 변경하시오.

```
UPDATE professors SET title='부교수' WHERE pname='오문환';
```

11) 교수 테이블에서 '전산'과 교수들의 급여를 10% 증가 시키시오.

```
UPDATE professors SET salary=salary + salary * 0.1 WHERE dept='전산';
```

12) 학생 테이블에서 '서연우' 학생의 학년을 2학년, 생년월일을 '74/10/06'로 변경하시오.

```
UPDATE students SET year=2, birthday='74/10/06' WHERE sname='서연우';
```

13) 강좌 테이블에서 모든 강좌 데이터의 최대 수강인원을 5, 수강 신청한 인원을 0으로 변경하시오.

```
UPDATE courses SET capacity=5, persons=0;
```

14) '오문환' 교수가 강의하는 강좌들의 강의실을 모두 '123'으로 변경하시오.

```
UPDATE courses SET room='123'  
WHERE instructor IN (SELECT pcode FROM professors WHERE pname='오문환');
```

```
SELECT * FROM courses;
```

15) 학생 테이블에서 '이재광'(228) 교수가 지도하는 학생들의 지도교수를 '이병렬'(221) 교수로 변경하시오.

```
SELECT * FROM students;  
UPDATE students SET advisor=(SELECT pcode FROM professors WHERE pname='이병렬')  
WHERE advisor=(SELECT pcode FROM professors WHERE pname='이재광');
```

```
SELECT * FROM students;
```

16) 수강신청 테이블에서 '파일처리론' 과목을 수강신청 한 학생들의 점수를 5점씩 감소시키시오.

```
UPDATE enrollments SET grade=grade-5  
WHERE lcode IN(SELECT lcode FROM courses WHERE lname='파일처리론');
```

17) 수강신청 테이블에서 '서연우'(92414029) 학생의 모든 과목 점수를 1점씩 감소시키시오.

```
UPDATE enrollments SET grade=grade+1  
WHERE scode IN (SELECT scode FROM students WHERE sname='서연우');
```

```
SELECT * FROM enrollments WHERE scode='92414029';
```

18) 수강신청 테이블에서 'C301' 강좌를 수강 신청한 인원수를 구하여 강좌 테이블 'C301' 수강인원수를 변경하시오.

```
UPDATE courses  
SET persons=(SELECT COUNT(*) FROM enrollments WHERE lcode='C301')  
WHERE lcode='C301';
```

```
SELECT * FROM courses;
```

19) 강좌 테이블에서 강좌별 수강인원수를 변경하시오.

```
UPDATE courses c  
SET persons=(SELECT COUNT(*) FROM enrollments e WHERE c.lcode=e.lcode)
```

```
SELECT * FROM courses;
```

20) 교수 테이블에 지도학생 수('stu_cnt')를 저장할 칼럼을 추가하고 학생 테이블에서 학생 수를 구하여 변경한다.

```
ALTER TABLE professors ADD stu_cnt INT DEFAULT 0;
```

```
UPDATE professors  
SET stu_cnt=(SELECT COUNT(*) FROM students WHERE advisor=pcode);
```

```
SELECT * FROM professors;
```

21) 교수 테이블에 담당강좌 수('cou_cnt')를 저장할 칼럼을 추가하고 강좌 테이블에서 강좌 수를 구하여 변경한다.

```
ALTER TABLE professors ADD cou_cnt INT DEFAULT 0;
```

```
UPDATE professors  
SET cou_cnt=(SELECT COUNT(*) FROM students WHERE instructor=pcode);
```

```
SELECT * FROM professors;
```

22) 학생 테이블에 수강신청 강좌 수('cou_cnt')를 저장할 칼럼을 추가하고 수강신청 테이블에서 강좌 수를 구하여 변경한다.

```
ALTER TABLE students ADD cou_cnt INT DEFAULT 0;
```

```
UPDATE students  
SET cou_cnt=(SELECT COUNT(*) FROM enrollments WHERE enrollments.scode=students.scode);
```

```
SELECT * FROM students;
```

23) 교수 테이블 일용일자 기본 값을 현재날짜로 변경한다.

```
DESC professors;
```

```
ALTER TABLE professors MODIFY hiredate DEFAULT SYSDATE;
```

24) 교수 테이블 학과의 문자 길이를 35자로, 기본 값을 '전산'으로 변경한다.

```
ALTER TABLE professors MODIFY DEPT VARCHAR(35) DEFAULT '전산';
```

25) 교수 테이블 'stu_cnt' 칼럼 이름을 'stu_count'로 변경한다.

```
ALTER TABLE professors RENAME COLUMN stu_cnt TO stu_count;
```

26) 교수 테이블 'stu_count' 칼럼을 삭제한다.

```
ALTER TABLE professors DROP COLUMN stu_count;
```

26) 수강신청 테이블의 이름을 enrollments에서 enroll로, enroll에서 enrollments로 변경한다.

```
ALTER TABLE enrollments RENAME TO enroll;
```

```
ALTER TABLE enroll RENAME TO enrollments;
```

• 학사관리 데이터베이스 뷰(VIEW)

VIEW를 생성하고 생성된 VIEW를 확인하고 생성된 VIEW를 삭제하는 SQL문은 아래와 같다.

```
-- VIEW 생성
CREATE 뷰이름 AS (SELECT 문장);

-- 생성된 VIEW 확인
SELECT * FROM user_views;

-- 생성된 VIEW 삭제
DROP VIEW 뷰이름
```

1) 학생 테이블에 모든 칼럼에 지도 교수이름을 추가하는 VIEW를 생성하여 '이병렬' 교수가 지도하는 학생 목록을 검색하시오.

```
INSERT INTO students(scode, sname) VALUES('92414030', '홍길동');
```

```
CREATE VIEW view_stu AS
SELECT s.*, pname FROM students s LEFT JOIN professors ON advisor=pcode;
```

```
SELECT * FROM view_stu WHERE pname='이병렬';
```

```
SELECT * FROM view_stu WHERE advisor IS NULL;
```

```
SELECT * FROM view_stu ORDER BY pname;
```

2) 강좌 테이블에 모든 칼럼에 담당 교수이름을 추가하는 VIEW를 생성하여 '이병렬' 교수가 담당하는 강좌 목록을 검색하시오.

```
INSERT INTO courses(lcode, lname) VALUES('E222', '회로이론');
```

```
CREATE VIEW view_cou AS
SELECT c.*, pname
FROM courses c LEFT JOIN professors ON instructor=pcode;
```

```
SELECT * FROM view_cou WHERE pname='이병렬';
```

```
SELECT * FROM view_cou WHERE pname IS NULL;
```

```
SELECT * FROM view_cou ORDER BY pname;
```

3) 수강신청 테이블 모든 칼럼에 강좌 이름을 추가하는 VIEW를 생성하여 '자료구조' 과목을 수강 신청한 학생 목록을 검색하시오.

```
CREATE VIEW view_enroll_cou AS
SELECT e.*, lname
FROM enrollments e, courses c
WHERE e.lcode=c.lcode;
```

```
SELECT * FROM view_enroll_cou WHERE pname='자료구조';
```

```
SELECT * FROM view_stu ORDER BY pname;
```

```
SELECT lcode, lname, MAX(grade), MIN(grade), TO_CHAR(AVG(grade), '999.99'), COUNT(*)
FROM view_enroll_cou
GROUP BY lcode, lname;
```

4) 수강신청 테이블 모든 칼럼에 학생 이름을 추가하는 VIEW를 생성하여 '서연우' 학생이 수강 신청한 강좌 목록을 검색하시오.

```
CREATE VIEW view_enroll_stu AS
SELECT e.*, sname
FROM enrollments e, students s
WHERE e.scode=s.scode;
```

```
SELECT * FROM view_enroll_stu WHERE sname='서연우';
```

```
SELECT * FROM view_enroll_stu ORDER BY sname;
```

```
SELECT scode, sname, MAX(grade), MIN(grade), TO_CHAR(AVG(grade), '999.99'), COUNT(*)
FROM view_enroll_stu
GROUP BY scode, sname;
```

5) 수강신청 테이블 모든 칼럼에 강좌 이름과 학생 이름을 추가하는 VIEW를 생성하여 '서연우' 학생이 수강 신청한 강좌 목록을 검색하시오.

```
CREATE VIEW view_enroll_cou_stu AS
SELECT e.lcode, lname, e.scode, sname, edate, grade
FROM enrollments e, courses c, students s
WHERE e.lcode=c.lcode AND e.scode=s.scode;
```

```
SELECT * FROM view_enroll_cou_stu WHERE sname='서연우';
```

6) 교수 테이블 모든 칼럼에 지도학생 수, 담당강좌 수를 추가하는 VIEW를 생성하고 지도학생 수가 3명 이상인 교수를 검색하시오.

```
CREATE VIEW view_pro2 AS
SELECT professors.*,
(SELECT COUNT(*) FROM students WHERE advisor=pcode) AS stu_cnt,
(SELECT COUNT(*) FROM courses WHERE instructor=pcode) AS cou_cnt
FROM professors;
```

```
SELECT * FROM view_pro2 WHERE stu_cnt >= 2;
```

7) 학생 테이블 모든 칼럼에 수강신청강좌 수, 학생 평균점수를 추가하는 VIEW를 생성하고 학생 평균점수가 높은 순으로 정렬하시오.

```
CREATE VIEW view_stu2 AS
SELECT s.*,
(SELECT COUNT(*) FROM enrollments e WHERE e.scode=s.scode) AS cou_cnt,
(SELECT TO_CHAR(AVG(grade), '999.99') FROM enrollments e WHERE e.scode=s.scode) AS stu_avg
FROM students s;
```

```
SELECT * FROM view_stu2 ORDER BY stu_avg DESC;
```

8) 강좌 테이블 모든 칼럼에 수강신청학생 수, 강좌 평균점수를 추가하는 VIEW를 생성하고 강좌 평균점수가 높은 순으로 정렬하시오.

```
CREATE VIEW view_cou2 AS
SELECT s.*,
(SELECT COUNT(*) FROM enrollments e WHERE e.scode=s.scode) AS stu_cnt,
(SELECT TO_CHAR(AVG(grade), '999.99') FROM enrollments e WHERE e.scode=s.scode) AS cou_avg
FROM students s;
```

```
SELECT * FROM view_cou2 ORDER BY cou_avg DESC;
```